Abstract

Python is quickly becoming an essential asset in every modern-day programmer's tool kit. Demand has been increasing ever since its launch and still continuing to grow. Statistics show that there has been an increase in python developers by 41% in the last 5 years. Essentially, this indicates that out of 25 million developers in the world, 8.2 million are Python developers. It is proof that python is a must-have skill for programmers, data scientists, and developers across many industries. (GeeksforGeeks [1])

This report focuses on extending a software development project initially designed for a prestigious football club using python as its main language interpreter. The existing program evaluates new players based on six crucial skills: Speed, Shooting, Passing, Defending, Dribbling, and Physicality, with ratings ranging from 0 to 5. The goal is to calculate an overall rating, employing a specific formular that yields a value between 0 and 100. This overall rating is then utilized to estimate a player's salary, referencing a provided dataset.

This program will then be extended to incorporate new functionality intended to enhance the program's robustness and utility. This includes refining the salary estimation process for specific overall rating ranges, ensuring the program handles inputs within the defined skill rating range, and implementing effective error handling mechanisms.

This paper also makes use of an additional tool called KIVY, an opensource multi-platform GUI development library for Python. This extension has the capacity to run on iOS, Android, Windows, OS X, and GNU/Linux making it a very versatile experimentational/ learning tool. As part of an indie project this project will be extended beyond this scope into an application that make use of innovative, multi-touch UI to be used across all devices. Objectively making the code reusable and deployable, allowing for quick and easy interaction design and rapid prototyping

Introduction

Python is a high-level language that is designed to be human readable and focuses on simplicity and readability. This allows developers to express concepts in fewer lines of code than might be possible in languages such as C++ or Java. Making python a general-purpose language suitable for a wide range of applications, including web development, data analysis, artificial intelligence, machine learning, automation, and more. (GeeksforGeeks [1])

Therefore, this paper will focus on extending the salary calculator for a football club using python. In the real world as a software engineer attention to detail is explicitly important. This means, meeting the requirements of the project brief as closely as our capacity will allow. Mastery of this skill will not only improve one's coding efficiency but will also ensure compatibility with the client's expectations or workflow. Objectively, this paper aims to encapsulate advanced functionalities essential for comprehensive football player evaluation. This will include an organized code structure, employing functions such as main(), calculate_rating(), and calculate_salary() for the purpose of enhancing modularity and readability. Similar to version 1 of this program, user interaction is prioritized, with the program prompting users to input player information. However, there are some key differences.one of which highlights that the program should accept data for precisely three players or halting if the user enters "end" instead of an ID.

Beyond skill assessment, the program dynamically calculates players' overall ratings, their corresponding salary ranges based on a provided dataset, and their ages derived from the inputted date of birth. Visualization tremendously assists decision making in any organization, there for the user experience will be further enhanced through a tabulated display of all data processed in a concise and organized overview. Furthermore, the program incorporates a feature to save this comprehensive table to a text file named players.txt, ensuring data persistence for future reference. This multifaceted program not only

streamlines the player evaluation process but also underscores the significance of clean code architecture and user-centric design principles. However, there are a few potential problems that could result from using a terminal-based interface as per the requirements. Part a software engineer's role includes problem solving. This requires in-depth thinking to develop a robust program with the least possibility of logic and user errors. Therefore, we have developed a solution an application that will address major real-world concerns. Some of which include a user-friendly interface, accessibility for non-technical users, reduced error threshold, Integration with operating systems and cross platform compatibility.

Methodology

The initiation phase began with the Gant Chart from Coursework 1 (Figure.1). The development process was broken down into a four-week process distributing the work into manageable objectives. The first week involved reevaluating the brief to ensure that the important details were noted. Once this part of the process was completed the pseudocode could then be refined to be used as a guide during this project. Pseudocode is generally referred to as method of writing up a set of instructions for a computer program using plain English. This is a good way of planning a program before coding (BBC Bitesize [2]).

With the guide in hand, formatting the code in the python interpreter became a frictionless process. Due to the structure of the previous code, It was relatively easy to move the main body into a dedicated function called main(). A function is a block of organized, reusable code that is used to perform a single, related action. (Tutorialspoint [3]). In this case we've defined out functions using the 'def' symbol. These functions provide better modularity for the application and a high degree of code reusing.

In most programs that employ functions, you will notice the use of "if __name__ == '__main__': ". Python files are called modules and they are identified by the .py file extension. A module can define various functions, classes, and variables. When the interpreter runs a module, the __name__ variable will be set as __main__ if the module that is being run is the main program. However, if the code is importing the module from another module the __name__ variable will be set to that module's name. (Python Software Foundation [4]).

Programs employ these types of modular structures are much more efficient and reusable.

Once the program is runs python initiates the imports of os, tabulate and datetime modules to be used within the code. These imports will later assist in file reading and writing, data presentation and validation methods for age input. Validating data indisputably makes a program more robust. Functions within the

code make use of type checks ‘str()’, ‘.isalpha’, ‘.lower()’ and ‘Try:’ to help streamline the data processing of the application.

While working code can be a goal, it is important to practice good coding habits. PEP 8, which stands for "Python Enhancement Proposal 8," is the official style guide for writing Python code. It was written by Guido van Rossum, Barry Warsaw, and Nick Coghlan and outlines conventions for writing readable and maintainable code. Adhering to PEP 8 ensures a consistent and standardized coding style across Python projects, making it easier for developers to collaborate and understand each other's code.

In the meticulous development of the football player evaluation program, the code underwent thorough debugging and validation processes. Code grade checks, such as the use of the footballer_skills module and the invocation of the advanced function, were employed to ensure the reliability and functionality of the code. Rigorous testing through code grade, including the execution of specified functions, provided a robust validation mechanism.

After completing all the project milestones we could then approach the next challenge, Deploying a functioning KIVY graphical user interface(GUI). Kivy is often used in an educational setting for teaching programming and app development. Its simplicity and cross-platform capabilities make it a suitable choice for to learn mobile and touch-based application development. (Kivy Developers [5]). This version of the application employs the use of classes and a wider range of libraries such as gridlayout, screen and App.

Findings & Discussion

The Python program, developed to automate player salary calculations, stands as a resilient and efficient solution, effectively tackling the challenges associated with traditional manual processes. This accomplishment is attributed to the program's modular design, where key functionalities are encapsulated within functions such as calculate_rating() and calculate_salary(). This design choice not only promotes code clarity but also ensures a maintainable codebase, enabling seamless updates and modifications in the future.

User interaction is a pivotal aspect of the program, and it excels through thoughtfully crafted prompts and robust input validation mechanisms. The Name_Check(message) function is a prime example of this user-centric approach, offering a friendly experience by validating player names and converting them to lowercase for consistency. Complementing this, the program incorporates a try-except structure, effectively catching ValueError instances, thereby preventing erroneous inputs and bolstering the overall reliability of the system.

Efficient data organization is achieved through the strategic use of lists, with ALL_PID, ALL_PFN, and ALL_DOB serving as ordered collections for player IDs, names, and dates of birth. This choice aligns seamlessly with the sequential and positional nature of player details entry, simplifying indexing and retrieval processes. While dictionaries present flexibility, the decision to utilize lists reflects a deliberate design tailored precisely to the program's specific requirements.

The program's prowess in presenting player information in a structured table format is facilitated by the tabulate library. Offering flexibility, the program provides two distinct tabulation options: P_Tab() and imported_P_Tab(). These not only enhance the display of player data but also allow for the saving of the generated table to a file named players.txt, demonstrating an efficient integration of the os library for

streamlined file operations. This dual functionality not only boosts the program's efficiency but also establishes a valuable record of player details for future reference.

Critical age calculations are handled by the verify_age() function. It leverages the capabilities of the datetime module. The concise arithmetic expression for age determination [age = today.year - dob_date.year - ((today.month, today.day) < (dob_date.month, dob_date.day))] ensures the accurate calculation of player ages based on the provided date of birth.

Furthermore, the inclusion of if __name__ == "__main__": refines the program's functionality. This construct ensures that the main code block, orchestrated by the main() function, is executed only when the script is run as the main program. This not only enhances the modular design by allowing functions to be defined and imported without triggering unintended executions but also promotes a clearer separation of concerns, contributing to a more organized and readable codebase.

In addition to the command-line interface (CLI) application, a Graphical User Interface (GUI) was developed using the Kivy framework, providing a more visually engaging and user-friendly experience for managing player salary calculations. A notable departure in this version is the incorporation of object-oriented programming principles, with the implementation of classes to encapsulate the GUI's components and functionalities.

The integration of Kivy and KivyMD libraries is evident through the inclusion of components such as GridLayout, Image, ScreenManager, Screen, Button, Label, BoxLayout, TextInput, MDDataTable, MDLabel, MDBoxLayout, ScrollView, MDRaisedButton, and Snackbar. These components contribute to the construction of an intuitive and dynamic interface, enhancing both accessibility and usability.

Theming capabilities, as indicated by ‘from kivymd.theming import ThemeManager’, ensure a consistent and professional appearance across various UI elements.

These UI Elements are then organized on their screens via the ScreenManager and Screen components, allowing users to seamlessly transition between different sections of the application, exemplified by self.manager.current = 'new_calculation_page'.

The GUI retains the critical data validation aspects present in the CLI application, with the iso_date_pattern regular expression ensuring the proper format of entered dates. The integration with the existing CLI logic is evident, as core functionalities such as calculating ratings, determining salary ranges, and validating data remain consistent between the CLI and GUI versions.

One notable feature is the dynamic table display enabled by the MDDataTable component, enhancing the visual representation of player information. The GUI's ability to present player details in a structured tabular format aligns with the program's existing capability.

Moreover, the inclusion of a Snackbar component enhances user feedback, providing informative messages or notifications within the application. This feature adds a layer of interactivity, keeping users informed about the status of their interactions.

In conclusion, the GUI implementation using Kivy and KivyMD enriches the player salary calculation program, offering an alternative interface with improved interactivity and aesthetics. The incorporation of classes introduces a more structured and modular approach, contributing to the program's maintainability and extensibility. This expansion into graphical interfaces signifies a commitment to enhancing user accessibility and usability, complementing the robust functionality provided by the CLI application.

Conclusion & Reflection

In conclusion, this project has demonstrated the versatility and power of Python in the realm of software development, particularly in the context of automating player salary calculations for a football club. Python's widespread adoption is evident in the growing number of developers worldwide, with a 41% increase in the last five years, making it an essential skill across various industries.

Initially the focus of the project was to extend a software development project for a football club, evaluating new players based on key skills and calculating their overall ratings and corresponding salaries. The program evolved through meticulous planning and coding, embracing good coding practices outlined in PEP 8. The modular design, encapsulating functionalities within functions like calculate_rating() and calculate_salary(), proved instrumental in achieving a maintainable and efficient codebase.

The user-centric design emphasized effective user interaction, data validation and the integration of error handling mechanisms, such as try-except structures which further enhanced the program's reliability. This coupled with efficient data organization, achieved through lists like ALL_PID, ALL_PFN, and ALL_DOB, facilitated seamless indexing and retrieval. The program's ability to present structured player information using the tabulate library, and save it to a file named players.txt, contributed to its reusability.

The introduction of the Kivy framework brought a graphical interface to the project, demonstrating its potential for enhancing user experience and accessibility. Object-oriented programming principles were embraced with the introduction of classes, making the code more modular and reusable. The integration of Kivy and KivyMD components, including MDDataTable and Snackbar, provided a dynamic and visually appealing alternative to the command-line interface.